"Either everything happens, or nothing does
-- that is the promise of a transaction."

MODULE 27

Transaction Management

Ensure reliable, consistent, and secure data operations in enterprise Spring applications with ACID guarantees, @Transactional, and disciplined rollback design.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 27 OVERVIEW

Ensure reliable, consistent, and secure data operations in enterprise Spring applications with ACID guarantees, @Transactional, and disciplined rollback design.

Either everything happens, or nothing does -- this module covers ACID properties, declarative @Transactional boundaries, propagation, and isolation, then proves rollback behavior in the Northstar CRM transfer lab.

DURATION & LAB

4 Hours Theory

ACID properties, @Transactional and AOP proxies, commit/rollback lifecycle, propagation, and isolation levels.

2 Hours Lab

Northstar CRM Transfers: move funds between sub-accounts and force a synthetic rollback.

Scenario

A debit without a matching credit is an incident -- all money movement must go through a @Transactional service.

MODULE ARC

Understand ACID

Atomicity, Consistency, Isolation, and Durability guarantee reliable multi-step operations.

Master @Transactional

Declarative management via AOP proxies -- and why self-invocation silently skips it.

Build the Lab

Implement TransferService, force ACC-FORCE-FAIL, and document before/after balances proving rollback.

KEY TAKEAWAY Total time: 4 hours theory + 2 hours hands-on lab. Correctly designed transactions protect data consistency and support reliable, enterprise-ready systems.

WHY TRANSACTIONS MATTER (1 OF 2)

Transactions ensure that a set of database operations either all succeed or all fail as a single unit.

This guarantees data integrity, consistency, and reliability -- especially in critical business processes.

TRANSACTIONS ARE ESSENTIAL FOR

Financial Operations

Transfers, payments, and reconciliations.

Order Processing

Create order, update inventory, charge customer.

Account Management

Create/update accounts and related details.

Reporting Accuracy

Consistent and reliable data for business decisions.

Data Integrity & Compliance

Meet regulatory requirements; protect customer data.

WITHOUT VS. WITH TRANSACTIONS

Scenario	Impact
Without Transactions	Partial updates can occur, leading to incorrect balances, data corruption, and loss of trust.
With Transactions	Your data remains accurate, consistent, and trustworthy -- even in the face of failures.

KEY TAKEAWAY Transactions are the foundation of reliable enterprise applications -- protecting your data and your customers.

WHY TRANSACTIONS MATTER (2 OF 2)

Real-world impact: the same $500 transfer, with and without a transaction guarantee.

SCENARIO WITHOUT TRANSACTIONS

1. **Account A: Debit $500**
   Balance $1,000 -> $500
2. **System Fails**
   Before credit completes
3. **Account B: No Credit**
   Balance stays $2,000 -- $500 is gone

SCENARIO WITH TRANSACTIONS

1. **Account A: Debit $500**
   Balance $1,000 -> $500
2. **All Operations Succeed**
   Debit + credit as one unit
3. **Account B: Credit $500**
   Balance $2,000 -> $2,500

Protects Data Integrity | **Ensures Customer Trust** | **Supports Reliable Business Operations**

KEY TAKEAWAY Transactions ensure 'all or nothing' execution -- critical for data integrity and business continuity.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to understand and apply transaction management to ensure data integrity, consistency, and reliability.

- **Understand Transaction Fundamentals —** define a transaction and explain why transactions are critical for data integrity in enterprise systems.
- **Explain ACID Properties —** describe Atomicity, Consistency, Isolation, and Durability and their role in reliable transactions.
- **Explore Spring Transaction Architecture —** understand how Spring manages transactions using PlatformTransactionManager and declarative transaction management.
- **Use @Transactional Effectively —** apply the @Transactional annotation to manage transactions declaratively in Spring applications.
- **Manage Transaction Lifecycle and Behavior —** understand the lifecycle of a transaction, including commit, rollback, and transaction boundaries.
- **Configure Propagation and Isolation —** configure and apply different propagation behaviors and isolation levels based on business requirements.
- **Identify Common Transaction Problems —** recognize issues such as deadlocks, timeouts, and inconsistent reads, and how to mitigate them.
- **Apply Best Practices in Transaction Management —** follow best practices for designing robust, scalable, and maintainable transactional applications.

KEY TAKEAWAY You will be able to design and implement reliable transactions that protect data consistency.

WHAT IS A TRANSACTION?

A transaction is a logical unit of work made up of one or more database operations.

The operations are treated as a single unit that must either all succeed (commit) or all fail (rollback). Transactions prevent partial updates and keep the database in a consistent state.

REAL-WORLD ANALOGY: A BANK TRANSFER

- Transferring money from Account A to Account B involves two operations: (1) debit money from Account A, (2) credit money to Account B.
- **Successful transaction —** both debit and credit happen; money is safely transferred.
- **Failed transaction —** if something goes wrong, neither happens; no money is lost.

A TRANSACTION IN ACTION

Path	Outcome
Commit (all succeed)	Changes are saved; database updated consistently.
Rollback (any fails)	All changes are undone; database remains unchanged.

KEY TAKEAWAY A transaction guarantees that either all operations complete, or none of them are applied.

REAL-WORLD BANKING EXAMPLE (1 OF 2)

Consider a funds transfer from Account A to Account B for $500 -- a single business process made of multiple database operations.

FUNDS TRANSFER -- STEP BY STEP

1. **Validate Request**
   Verify accounts exist, check balance
2. **Debit Account A**
   Subtract $500 from balance
3. **Record Debit**
   Insert ledger entry
4. **Credit Account B**
   Add $500 to balance
5. **Record Credit**
   Insert ledger entry

KEY IDEA

Either all five steps are completed successfully, or none of them are applied. This ensures accurate balances and prevents money from appearing or disappearing.

KEY TAKEAWAY These steps are executed as a single transaction to maintain data integrity.

Real-World Banking Example

Consider a fund transfer of $500 from Account A to Account B.

THE TRANSACTION

A single logical unit of work with multiple steps

1. Debit
Account A

Subtract $500 from Account A

2. Credit
Account B

Add $500 to Account B

3. Record
Transaction

Writetransaction to database log

4. Update
Balance

Update account balances

ALL OR NOTHING

Either all 4 steps complete successfully (COMMIT) or none of the steps are applied (ROLLBACK).

WITHOUT TRANSACTIONS

If something goes wrong in Step 2 or 3:

Problem 1: After Step 1 Fails
Account A is debited, but Account B is not credited.

Account A	≠	Account B
$9,500		$5,000

Problem 2: After Step 2 Fails
Account B is credited, but record is not logged.

Account A	≠	Account B
$9,500		$5,500

Result
Data becomes inconsistent, leading to mismatched balances and loss of trust.

Transactions Ensure
Consistency, reliability, and trust in every financial operation.

In banking, transactions guarantee that money is always correct—every time.

REAL-WORLD BANKING EXAMPLE (2 OF 2)

Success (commit) and failure (rollback) outcomes for the $500 transfer, and why it matters.

Path	Account A Balance	Account B Balance
Success -- Commit	$1,000 - $500 = $500	$2,000 + $500 = $2,500
Failure -- Rollback	$1,000 (no change)	$2,000 (no change)

WHAT HAPPENS ON EACH PATH

- **Success -- Commit —** all operations complete successfully; changes are permanently saved; the database is updated consistently.
- **Failure -- Rollback —** if any step fails (network issue, system error), the entire transaction is rolled back; no changes are saved; data remains consistent.

Protects Data Integrity **Ensures Customer Trust** **Supports Reliable Business Operations**

KEY TAKEAWAY Transactions protect customer money, ensure accurate reporting, and maintain trust in banking systems.

ACID PROPERTIES OVERVIEW

ACID is a set of four properties that guarantee reliable transaction processing in database systems.

Together, they ensure data remains accurate, consistent, and trustworthy even in the event of failures or concurrent access.

Atomicity

All operations succeed, or none are applied. All or nothing.

Consistency

The database moves from one valid state to another. Always valid.

Isolation

Concurrent transactions do not interfere with each other.

Durability

Once committed, results are permanently saved. Permanent changes.

Property	One-Line Example
Atomicity	If a transfer fails after debiting Account A but before crediting Account B, the debit is rolled back.
Consistency	A transaction will not violate rules such as 'account balance cannot be negative.'
Isolation	When two users transfer money at the same time, each sees a consistent view without interference.
Durability	After a successful transfer and commit, even a server crash does not lose the updated balances.

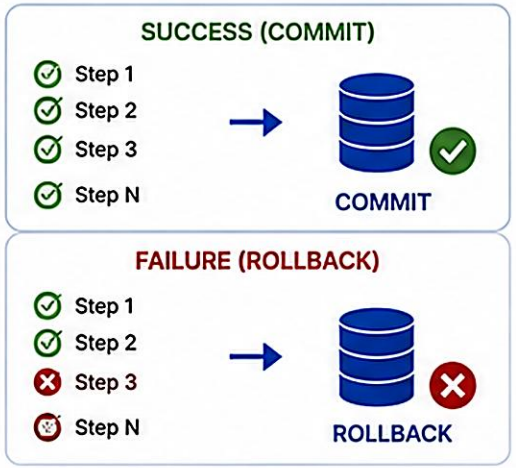
KEY TAKEAWAY ACID properties are the foundation of reliable transactions: correct, consistent, isolated, and permanent.

ACID Properties Overview

ACID properties guarantee that database transactions are reliable, consistent, and safe, ensuring data integrity even in the face of errors or failures.

A ATOMICITY
All or nothing.

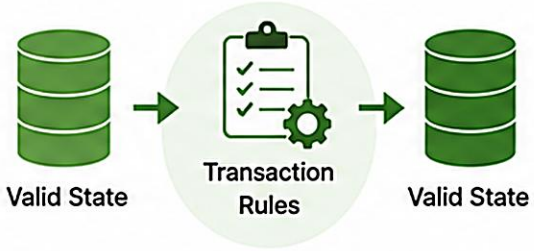
A transaction is treated as a single unit of work. Either all operations succeed or none are applied.



Example
If any step in a fund transfer fails, no money is transferred.

C CONSISTENCY
Always valid.

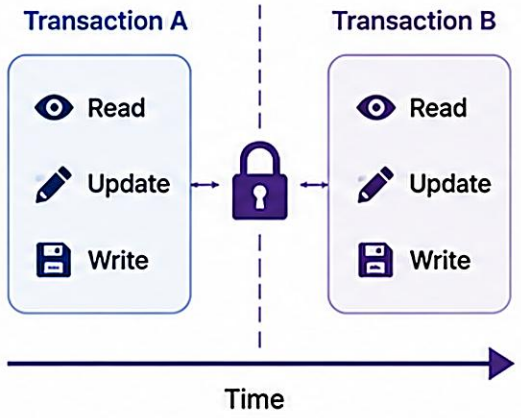
A transaction brings the database from one valid state to another, preserving all rules and constraints.



Example
Account balances never go negative. Constraints (e.g., min balance, unique ID) are always enforced.

I ISOLATION
Separated operations.

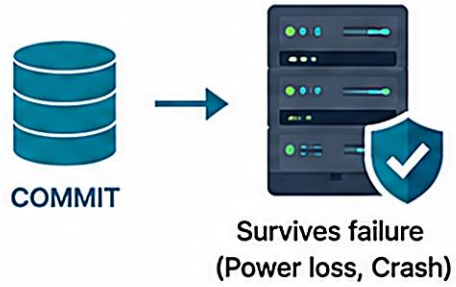
Concurrent transactions do not interfere with each other. Each transaction appears isolated.



Example
Two users transferring funds at the same time do not see each other's intermediate changes.

D DURABILITY
Permanent results.

Once a transaction is committed, the results are permanent, even in case of system failure.



Example
After a successful transfer, updated balances remain, even if the system crashes.

ACID properties ensure reliable transactions that maintain data integrity and build trust in enterprise systems.

ATOMICITY

Atomicity ensures that all operations within a transaction are treated as a single, indivisible unit of work.

Either all operations are completed and committed, or none are applied at all. There are no partial updates.

EXAMPLE: $500 BANK FUNDS TRANSFER

Path	Success (All or Nothing)	Failure (Rollback)
Result	All operations succeed; transaction commits; changes are saved.	Any operation fails; entire transaction rolls back; nothing changes.

KEY TAKEAWAY Atomicity guarantees that a transaction either fully succeeds or leaves the system exactly as it was before it started.

Atomicity

All or Nothing

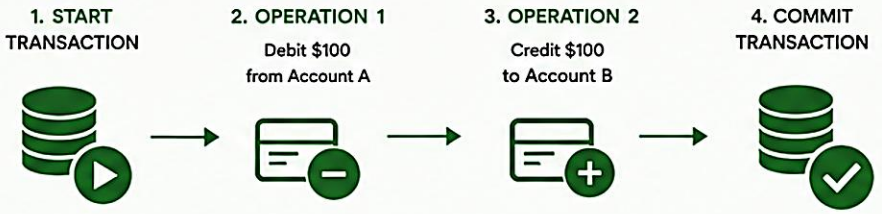
Atomicity guarantees that partial updates never happen.
All or Nothing.

Atomicity ensures that a transaction is treated as a single unit of work.
Either all operations in the transaction are completed successfully, or none are.

WHY IT MATTERS

- Prevents data inconsistencies
- Protects integrity of businees operations
- Ensures reliability in critical systems
- Builds trust in the application and database

ATOMIC TRANSACTION – SUCCESS (ALL OPERATIONS COMPLETE)



FINAL RESULT

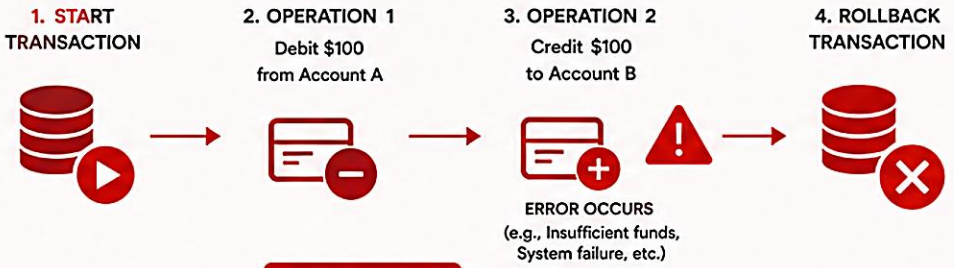
Before Transaction	
Account	Balance
Account A	$1,000
Account B	$500

→

After Transaction (Committed)	
Account	Balance
Account A	$900
Account B	$600

Both operations succeed.
Transaction is committed.

ATOMIC TRANSACTION – FAILURE (ALL OPERATIONS ROLLED BACK)



FINAL RESULT

Before Transaction	
Account	Balance
Account A	$1,000
Account B	$500

→

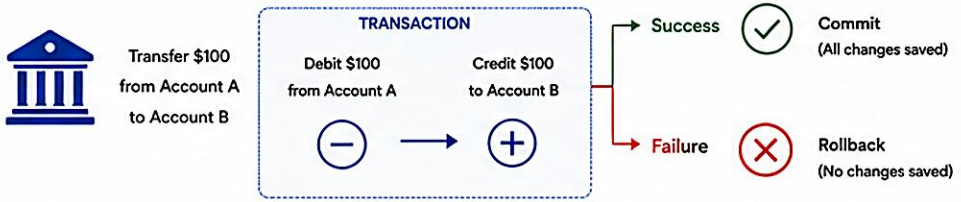
After Transaction (Rolled Back)	
Account	Balance
Account A	$1,000
Account B	$500

One operation failed.
Entire transaction is rolled back.

KEY POINTS

- A transaction is indivisible.
- Partial execution is not allowed.
- If any step fails, all changes are undone.
- Ensures consistent state of the database.

BANKING EXAMPLE



RELATED TO ACID

A	C	I	D
Atomicity	Consistency	Isolation	Durability
All or Nothing	Valid State	No Interference	Permanent

REMEMBER Atomicity is the foundation of reliable transactions. It ensures that your data is never left in a half-updated or inconsistent state.

CONSISTENCY

Consistency ensures a transaction brings the database from one valid state to another, obeying all business rules and constraints.

It guarantees that the database never ends up in an invalid state -- every committed transaction leaves the data accurate and rule-compliant.

EXAMPLE: BEFORE AND AFTER A $500 TRANSFER

Metric	Before Transaction	After Transaction (Valid State)
Account A Balance	$1,000	$500
Account B Balance	$2,000	$2,500
Total Money in the Bank	$3,000	$3,000 -- unchanged

CONSISTENT VS. INCONSISTENT TRANSACTIONS

- **Consistent (valid state) —** all rules are followed; total money in the bank never changes; the database remains valid.
- **Inconsistent (invalid state) —** a rule is violated or a constraint fails (e.g. a negative balance); the change is rejected.

KEY TAKEAWAY Consistency is about correctness -- every committed transaction leaves the database valid and accurate.

Consistency

Always in a Valid State

Consistency ensures that a transaction brings the database from one **valid state** to another **valid state**, preserving all defined rules, constraints, and business logic.

Consistency guarantees that data integrity rules are never violated. **All constraints must hold true** before and after a transaction.

WHY IT MATTERS

- Maintains accurate and reliable data
- Enforces business rules and constraints
- Prevents invalid or corrupt data
- Builds trust in the system and data

CONSISTENT TRANSACTION – MAINTAINS VALID STATE

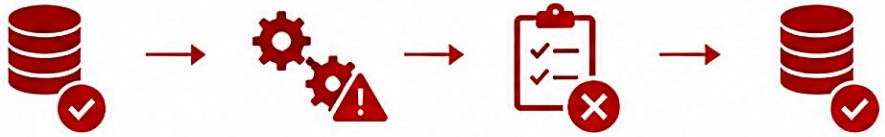
1. BEFORE TRANSACTION (Valid State)
2. PERFORM OPERATIONS (Follow Business Rules)
3. VALIDATE CONSTRAINTS (All Rules Satisfied)
4. AFTER TRANSACTION (Valid State)

EXAMPLE CONSTRAINTS ENFORCED

Primary Key	Foreign Key	CHECK Constraint	NOT NULL	Business Rules
Must be Unique	Must Exist	Must be True	Cannot be Null	Must be Followed

INCONSISTENT TRANSACTION – VIOLATES RULES

1. BEFORE TRANSACTION (Valid State)
2. PERFORM OPERATIONS (Violates a Rule)
3. VALIDATE CONSTRAINTS (Rule Violated)
4. TRANSACTION ROLLED BACK (Revert to Valid State)



EXAMPLE OF VIOLATION

Business Rule: Account balance cannot be negative.

Debit $200 from Account A (Balance $100) → Balance becomes -$100 (Invalid State) → **Rule Violated!** Transaction Rolled Back

BANKING EXAMPLE

Initial Valid State

Account	Balance
Account A	$1,000
Account B	$500

Transfer $200 from Account A to Account B

After Transaction (Consistent State)

Account	Balance
Account A	$800
Account B	$700

All rules satisfied. Data remains consistent.

KEY POINTS

- Database must be in a valid state before the transaction.
- Transaction must follow all defined rules and constraints.
- If any rule is violated, the transaction is rolled back.
- Ensures data accuracy and reliability across the system.

COMMON CONSTRAINTS

- **Primary Key** – Ensures uniqueness of records
- **Foreign Key** – Ensures referential integrity
- **CHECK** – Ensures values meet specific conditions
- **NOT NULL** – Ensures required data is present
- **Business Rules** – Domain rules must always hold true

RELATION TO ACID

A	C	I	D
Atomicity	Consistency	Isolation	Durability
All or Nothing	Valid State	No Interference	Permanent

TAKEAWAY

Consistency ensures your data remains accurate, valid, and trustworthy. Every transaction must leave the database in a correct and valid state.

ISOLATION

Isolation ensures that concurrent transactions do not interfere with each other -- each appears to run alone.

Intermediate changes made by one transaction are not visible to other transactions until it is committed.

EXAMPLE: TWO USERS TRANSFERRING AT ONCE

- T1 (User A) reads Account balance = $1,000, prepares to debit $300, then commits.
- T2 (User B) reads the same account. Isolation ensures T2 cannot see T1's uncommitted change -- T2 reads $1,000, not $700, until T1 commits.
- Both transactions run as if they are the only one using the database.

WITHOUT ISOLATION -- PROBLEMS

Problem	What Happens
Dirty Read	T2 reads data T1 wrote but has not committed; if T1 rolls back, T2 read invalid data.
Non-Repeatable Read	T2 reads the same row twice, gets different results because T1 committed changes in between.
Phantom Read	T2 re-runs a query and gets a different set of rows because T1 inserted or deleted rows.

Analogy: think of two accountants working in separate rooms on the same ledger. Neither sees the other's updates until one finalizes and shares the results.

KEY TAKEAWAY Isolation prevents dirty reads, non-repeatable reads, and phantom reads until a transaction commits.

Isolation

Transactions Don't Interfere with Each Other

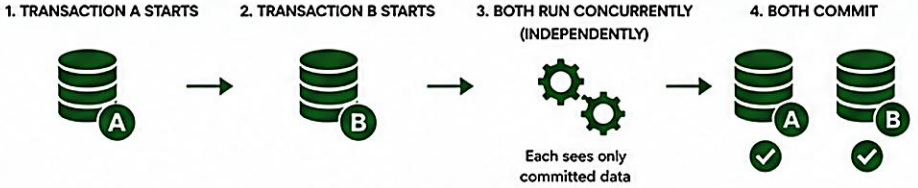
Isolation ensures that concurrent transactions are executed independently so that the intermediate results of one transaction are not visible to others until it is committed.

Isolation guarantees that concurrent transactions do not affect each other. Each transaction behaves **as if it is running alone.**

WHY IT MATTERS

- Prevents dirty reads, non-repeatable reads, and phantom reads
- Ensures accurate and predictable results in concurrent environments
- Maintains data integrity in multi-user systems
- Critical for financial systems, order processing, and reservations

ISOLATION IN ACTION – TRANSACTIONS DO NOT INTERFERE



EXAMPLE: BANK ACCOUNTS

Initial Balance (Committed)

Account	Balance
Account A	$1,000
Account B	$500

Transaction A: Transfer $100 from A to B →

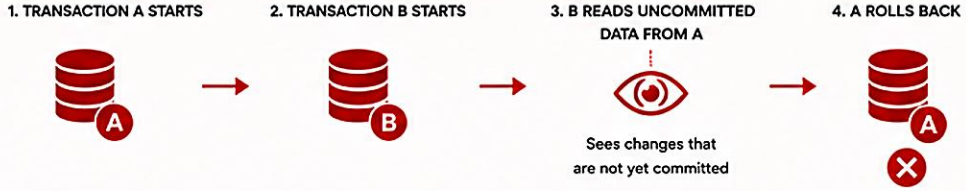
Transaction B: Transfer $200 from B to A ←

After Both Commit

Account	Balance
Account A	$1,100
Account B	$400

Both transactions completed without interfering with each other.

LACK OF ISOLATION – TRANSACTIONS INTERFERE



EXAMPLE: DIRTY READ

Initial Balance (Committed)

Account	Balance
Account A	$1,000
Account B	$500

Transaction A: Transfer $100 from A to B (UNCOMMITTED) →

Transaction B: Reads A = $900 (Dirty Read) and makes decision →

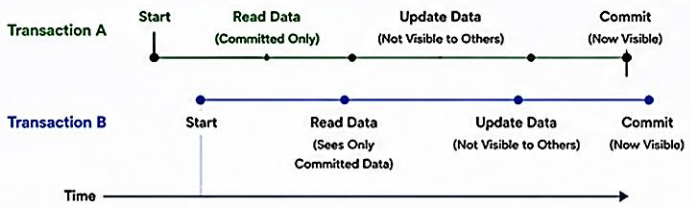
Transaction A ROLLS BACK: Changes Discarded →

Final State (After Rollback)

Account	Balance
Account A	$1,000
Account B	$500

Transaction B made a decision based on uncommitted (dirty) data. Data became inconsistent.

CONCURRENT TRANSACTION TIMELINE (WITH ISOLATION)



With proper isolation, each transaction works with a consistent snapshot of the data and other transactions do not see intermediate changes.

ISOLATION LEVELS (SQL STANDARD)

Isolation Level	Prevents
READ UNCOMMITTED	Dirty Reads
READ COMMITTED	Dirty Reads
REPEATABLE READ	Dirty Reads, Non-Repeatable Reads
SERIALIZABLE	Dirty Reads, Non-Repeatable Reads, Phantom Reads

COMMON ANOMALIES PREVENTED BY ISOLATION

Dirty Read
Reading data written by another transaction that has not been committed.

Non-Repeatable Read
Reading the same row twice and getting different results because another transaction modified and committed it.

Phantom Read
Re-executing a query returns additional rows because another transaction inserted new rows and committed.

BEST PRACTICES

- Choose the appropriate isolation level based on business requirements.
- Higher isolation = higher consistency but may reduce performance.
- Use the lowest isolation level that ensures correctness for your use case.
- Test concurrent scenarios to validate isolation behavior.

RELATION TO ACID

A	C	I	D
Atomicity	Consistency	**Isolation**	Durability
All or Nothing	Valid State	**No Interference**	Permanent

KEY TAKEAWAY

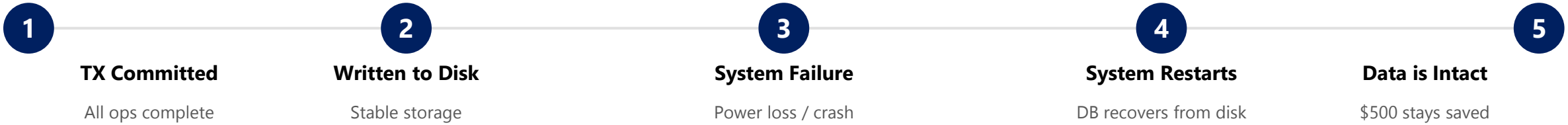
Isolation ensures that concurrent transactions do not interfere with each other. Each transaction sees a consistent view of the data, leading to accurate, reliable, and predictable outcomes in multi-user systems.

DURABILITY

Durability ensures that once a transaction has been committed, its changes are permanently saved and survive system failures.

Committed data is permanent and cannot be lost -- even in the face of power loss, crashes, or hardware failures.

EXAMPLE: FUNDS TRANSFER WITH SYSTEM FAILURE



Scenario	Before Failure	After Recovery
With Durability	A: $500 · B: $2,500 (committed)	A: $500 · B: $2,500 -- data persisted on disk
Without Durability	A: $500 · B: $2,500 (not written to disk)	A: $1,000 · B: $2,000 -- committed changes lost

KEY TAKEAWAY Durability guarantees that once data is committed, it remains safe and persistent no matter what happens next.

Durability

Committed Data is Permanent

Durability ensures that once a transaction is committed, its changes are permanently saved and will survive system failures, crashes, or power outages.

Durability guarantees that committed transactions are never lost—once data is saved, it remains safe even in case of system failure.

Commit = Permanent.

WHY IT MATTERS

- Protects data against system crashes or power failures
- Ensures long-term reliability and business continuity
- Critical for financial transactions, orders, bookings, and records
- Builds trust and confidence in the system

DURABILITY IN ACTION – DATA IS PERMANENT AFTER COMMIT

1. TRANSACTION EXECUTES — Transaction updates data
2. DATA WRITTEN TO MEMORY — Changes are stored in memory (RAM)
3. DATA FLUSHED TO DISK (LOG) — Changes are written to stable storage (WAL/Log)
4. COMMIT SUCCESSFUL — Transaction is committed
5. DATA PERSISTENT AFTER FAILURE — Data remains even after system restart

EXAMPLE: BANK DEPOSIT (DURABLE COMMIT)

Before Transaction	
Account	Balance
Account A	$1,000
Account B	$500

Transfer $200 from A to B → Commit Successful → Disk / Log Persisted → System Crash → System Restart

After Restart (Data Intact)	
Account	Balance
Account A	$800
Account B	$700

Even after a crash, the committed data is permanent and intact.

WITHOUT DURABILITY – DATA CAN BE LOST

1. TRANSACTION EXECUTES — Transaction updates data
2. DATA IN MEMORY — Changes are stored in memory (RAM)
3. SYSTEM CRASHES — Crash occurs before data is saved to disk
4. DATA LOST (NOT DURABLE) — Uncommitted or not persisted data is lost
5. SYSTEM RESTARTS — Database restores previous state

EXAMPLE: BANK DEPOSIT (DATA LOST DUE TO CRASH)

Before Transaction	
Account	Balance
Account A	$1,000
Account B	$500

Transfer $200 from A to B → Crash Before Commit → Restart System

After Restart (Data Lost)	
Account	Balance
Account A	$1,000
Account B	$500

Since changes were not persisted, the transaction is lost and data is not durable.

HOW DURABILITY IS ACHIEVED

Write-Ahead Logging (WAL)
All changes are written to a log on disk before they are applied to the database.

Synchronous Commit
Transaction is committed only after the log is safely written to stable storage.

Replication & Backup
Data is replicated and backed up to protect against hardware or site failures.

ACID Guarantee
Once committed, data will survive crashes, restarts, and power outages.

DURABILITY GUARANTEE SCENARIOS

Scenario	Data After Restart
Power Failure After Commit	Data is retained
System Crash After Commit	Data is retained
Disk Failure (with replication)	Data is retained
Before Commit (Crash)	Data is not saved

REAL-WORLD EXAMPLE

When you transfer money in online banking and the system commits the transaction, the money will be there even if the bank's server crashes immediately after.

Durability ensures your important data is never lost after a successful commit.

BEST PRACTICES

- Use a reliable database with Write-Ahead Logging.
- Ensure logs are written to non-volatile storage.
- Enable replication and regular backups.
- Test recovery procedures and disaster scenarios.
- Monitor storage health and system stability.

RELATION TO ACID

A	C	I	D
Atomicity	Consistency	Isolation	Durability
All or Nothing	Valid State	No Interference	Permanent

KEY TAKEAWAY

Durability is the guarantee that once data is committed, it is permanent and will never be lost, even in the event of failures. It ensures long-term reliability and data protection.

TRY IT YOURSELF -- EXERCISE 1: ACID FOR CRM TRANSFERS

Analysis exercise -- tie each ACID letter to a real Northstar CRM transfer observation, not a textbook definition.

Goal: for each ACID letter, write one sentence grounded in real Northstar CRM transfer accounts -- not a restated definition.

STEP-BY-STEP

Step	What To Do
1 -- Fill ACID	In notes/acid-crm.md, write one CRM sentence per ACID letter (Atomicity, Consistency, Isolation, Durability).
2 -- Check the reference	Align: Atomicity = debit+credit+log all succeed or none; Consistency = balances never violate invariants; Isolation = concurrent transfers never see half-updates; Durability = the committed log survives a restart.
3 -- List the accounts	Name all four accounts you'll reference: ACC-1001-MAIN, ACC-1001-LOYALTY, ACC-1002-MAIN, and the synthetic force id ACC-FORCE-FAIL.
4 -- State the boundary	Write one line distinguishing scope: this pre-lab explains ACID in words; Lab 27 proves rollback with real @Transactional code.

PASS CRITERIA

- All four ACID letters explained in CRM-specific language, not generic definitions.
- The synthetic force-fail account (ACC-FORCE-FAIL) is listed alongside the three real accounts.
- The pre-lab vs. Lab 27 boundary is stated clearly: words now, code proof later.

KEY TAKEAWAY TRY IT NOW -- complete Exercise 1 (exercises/exercise-01-acid-crm.md) before continuing: four ACID sentences, four accounts named, one boundary line.

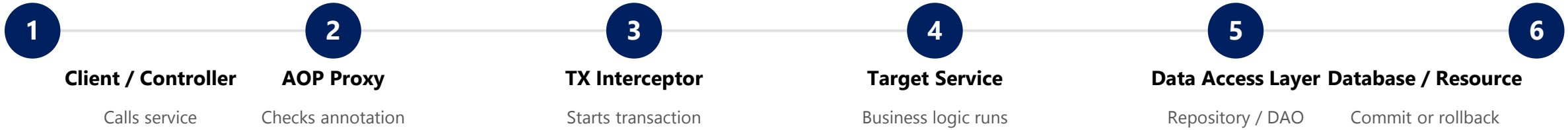
SPRING TRANSACTION ARCHITECTURE (1 OF 2)

Spring provides a declarative transaction management abstraction that simplifies transaction handling across your application layers.

KEY COMPONENTS

- **@Transactional —** declarative annotation defining transaction boundaries.
- **AOP Proxy —** creates a proxy around Spring beans to intercept transactional methods.
- **PlatformTransactionManager —** manages begin, commit, and rollback using the underlying resource.
- **Data Source / Resource —** the actual resource (JDBC, JPA, JMS) where data is persisted.

SPRING TRANSACTION FLOW

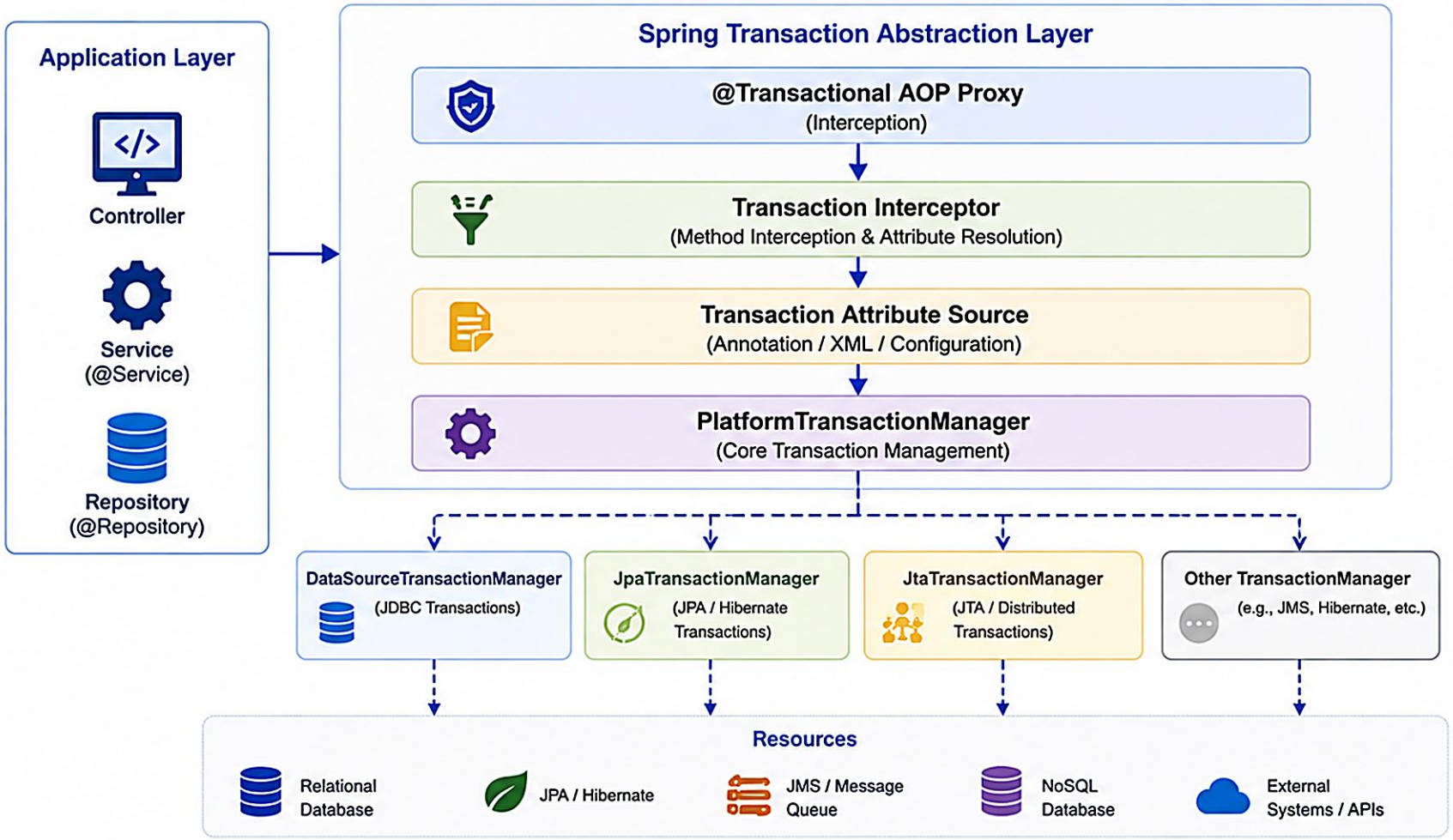


Centralized Management **Declarative** **Consistent & Reliable** **Enterprise Ready**

KEY TAKEAWAY Spring uses AOP proxies and PlatformTransactionManager to deliver ACID guarantees with almost no boilerplate.

Spring Transaction Architecture

Spring provides a consistent abstraction for transaction management
that works with different data access technologies and transaction managers.



How it Works

1. A method annotated with @Transactional is called from the application.
2. The AOP proxy intercepts the call and invokes the Transaction Interceptor.
3. The interceptor resolves transaction attributes (propagation, isolation, timeout, readOnly, rollback rules).
4. The appropriate PlatformTransactionManager is chosen and transaction is started.
5. On method completion:
   - Success → **Commit**
   - Exception → **Rollback**

Key Points

- Consistent programming model across all data access technologies.
- Pluggable transaction managers.
- Declarative transactions via @Transactional.
- Supports local and distributed transactions.

Spring's transaction architecture provides a clean abstraction that separates application code from transaction management concerns, enabling flexibility, consistency, and reliability.

SPRING TRANSACTION ARCHITECTURE (2 OF 2)

How the flow works step by step, and what happens on the success and exception paths.

HOW IT WORKS, STEP BY STEP

- 1. Client calls a service method.
- 2. The AOP proxy intercepts the call and checks for @Transactional.
- 3. The transaction interceptor starts a transaction before method execution, using PlatformTransactionManager.
- 4. The target service executes its business logic; data access happens via Repository/DAO.
- 5. The data access layer performs database operations using EntityManager or JDBC.
- 6. On success, the transaction is committed; on failure, it is rolled back.
- 7. Control returns to the AOP proxy, and then back to the client.

OUTCOME

Path	What Happens
Success	Transaction committed; all changes saved (Durability); ACID properties are guaranteed.
Exception / Failure	Transaction rolled back; no changes are saved; the system remains consistent.

KEY TAKEAWAY Spring's declarative transaction management works consistently across JDBC, JPA, JMS, and more.

@TRANSACTIONAL ANNOTATION (1 OF 2)

@Transactional is a declarative annotation in Spring that tells Spring to automatically start, commit, or roll back a transaction.

WHERE TO USE

- **At Method Level —** applied to a specific method; only that method runs in a transaction.
- **At Class Level —** applied to the entire class; all public methods run in a transaction unless overridden.

DEFAULT BEHAVIOR

- Starts a transaction before method execution.
- Commits if the method completes successfully.
- Rolls back if a RuntimeException or Error occurs (checked exceptions do NOT roll back by default).
- Uses default propagation = REQUIRED, isolation = DEFAULT, readOnly = false.

Method-Level @Transactional

```
@Service
public class TransferService {

  @Transactional
  public void transfer(String fromId, String toId,
                       BigDecimal amount) {
    accounts.debit(fromId, amount);
    accounts.credit(toId, amount);
  }
}
```

Class-Level @Transactional

```
@Service
@Transactional
public class AccountService {
  public void debit(String id, BigDecimal amt) { /* ... */ }
  public void credit(String id, BigDecimal amt) { /* ... */ }
}
```

KEY TAKEAWAY @Transactional is declarative -- but by default it only rolls back on unchecked exceptions.

@TRANSACTIONAL ANNOTATION (2 OF 2)

Common configurable attributes, the exception/rollback example, and best practices -- including the self-invocation trap.

COMMON ATTRIBUTES

- **propagation —** defines how the transaction behaves with an existing transaction.
- **isolation —** defines the isolation level of the transaction.
- **timeout —** sets the maximum time (seconds) for the transaction to complete.
- **readOnly —** hints that the transaction is read-only; may improve performance.
- **rollbackFor / noRollbackFor —** defines which exceptions should (or should not) trigger rollback.

BEST PRACTICES

- Place @Transactional on service-layer methods.
- Keep transactions short and focused; avoid self-invocation (calling another @Transactional method from within the same class skips the proxy).

Exception & Rollback Example

```
@Transactional
public void withdraw(String id, BigDecimal amount) {
  accounts.debit(id, amount);
  if (amount.compareTo(BigDecimal.ZERO) > 0
      && insufficientFunds(id)) {
    throw new InsufficientFundsException(id);
  }
}
// Result: transaction rolled back.
// No changes are saved to the database.
```

SELF-INVOCATION PITFALL

Calling this.someTxMethod() from inside the same class bypasses the AOP proxy -- @Transactional on that method is silently ignored. Always call through an injected bean.

KEY TAKEAWAY Configure propagation, isolation, timeout, and rollback rules deliberately -- self-invocation skips them all.

TRY IT YOURSELF -- EXERCISE 2: TRANSACTION BOUNDARY PLACEMENT

Architecture exercise -- decide exactly where the transfer transaction boundary belongs, and defend the choice.

Goal: choose the correct home for the transfer's transaction boundary -- service, controller, or repository -- and order the steps inside it.

STEP-BY-STEP

Step	What To Do
1 -- Choose	In notes/tx-boundary.md, pick the boundary for debit+credit+log: TransferService.transfer(...), the controller method, or the repository.
2 -- Check the reference	Confirm: TransferService.transfer(...) is preferred; a controller method should be avoided; repository-only is too narrow for a multi-step business operation.
3 -- Order the internal steps	Write the order inside the boundary: load accounts -> debit -> credit -> write TransactionLog.
4 -- Note the correlation	Record that the happy-path evidence uses correlation id lab-request-001.

WHERE @TRANSACTIONAL BELONGS

TransferService.java vs. TransferController.java

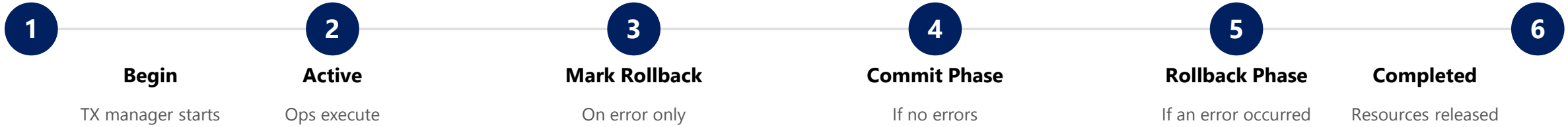
```
// Correct -- the service owns the transaction boundary
@Service
public class TransferService {
  @Transactional
  public TransactionLog transfer(String fromId, String toId, BigDecimal amount) { /* debit, credit, log */ }
}

// Wrong -- controllers must stay thin; @Transactional does NOT belong here
@RestController public class TransferController { /* delegates to TransferService */ }
```

KEY TAKEAWAY TRY IT NOW -- complete Exercise 2 (exercises/exercise-02-transaction-boundary.md): service chosen over controller, four steps ordered, correlation id noted.

TRANSACTION LIFECYCLE (1 OF 2)

A transaction goes through a well-defined lifecycle from start to completion, handled automatically by Spring and the transaction manager.

THE TRANSACTION LIFECYCLE



KEY POINTS

- Transactions ensure data integrity by grouping multiple operations as a single unit of work.
- Either all operations succeed (COMMIT) or all are undone (ROLLBACK).
- The transaction manager controls the lifecycle automatically.
- You focus on business logic -- Spring handles the transaction details.

KEY TAKEAWAY Understanding the transaction lifecycle helps you write more reliable, data-safe applications.

Transaction Lifecycle

How a transaction is created, used and completed in a Spring application

KEY PARTICIPANTS

Application Client
Calls the @Transactional method.

Spring AOP Proxy
Intercepts the method call and manages transaction boundaries.

Transaction Manager
Creates, commits, rolls back and manages the transaction.

Resource (DB, JMS, etc.)
Resources that participate in the transaction.

Transactional Component
The service/repository containing business logic.

1 Method Call Intercepted

Proxy

A client calls a method annotated with @Transactional. The Spring AOP proxy intercepts the call.

2 Transaction Creation

Transaction Manager creates a new transaction (if none exists) and binds it to the current thread.

Status: ACTIVE

3 Resource Acquisition

Resources (e.g., DataSource) obtain a connection and associate it with the current transaction.

4 Business Logic Execution

The target method executes business logic within the transaction boundary.

5 Transaction Completion (Commit/Rollback)

If execution is successful, the transaction is committed.

If an exception occurs, it is marked for rollback.

6 Resource Release

Connections and other resources are released and unbound from the thread.

7 Transaction Closed

Transaction is completed. The thread is cleaned up and control returns to the caller.

Begin — Transaction Active — Transaction Completed — End

COMMIT FLOW (Success Path)

Business Logic Success → validate & prepare commit → flush changes to database → commit transaction → Release Resources → Transaction Closed

ROLLBACK FLOW (Failure Path)

Exception Thrown → mark transaction for rollback → discard changes (rollback) → Release Resources → Transaction Closed

WHEN IS A TRANSACTION ROLLED BACK?

- RuntimeException (unchecked) by default
- Error
- Custom exceptions if configured (rollbackFor)

PROPAGATION BEHAVIORS (Common)

REQUIRED	Join existing or create new
REQUIRES_NEW	Always create a new transaction
SUPPORTS	Execute non-transactionally if none
NOT_SUPPORTED	Suspend transaction if exists
MANDATORY	Must run in a transaction
NEVER	Must not run in a transaction

ISOLATION LEVELS (Common)

READ_UNCOMMITTED	Lowest isolation
READ_COMMITTED	Prevents dirty reads
REPEATABLE_READ	Prevents non-repeatable reads
SERIALIZABLE	Highest isolation

KEY BENEFITS

Data Integrity
Ensures ACID properties.

Consistency
All changes succeed or none.

Concurrency Control
Manages isolation and locking.

Simplified Development
Spring manages transactions for you.

BEST PRACTICES

- ✓ Keep transactions short and focused.
- ✓ Do not perform I/O or long running tasks inside a transaction.
- ✓ Choose the right propagation and isolation level.
- ✓ Handle exceptions properly to avoid unexpected rollbacks.
- ✓ Use read-only transactions for query operations (@Transactional(readOnly=true)).

EXAMPLE

```
@Transactional(rollbackFor = Exception.class,
               isolation = Isolation.READ_COMMITTED)
public void transferMoney(...) {
    // business logic
}
```

TRANSACTION LIFECYCLE (2 OF 2)

The success path versus the error path, and what happens behind the scenes on every transaction.

SUCCESS PATH -- COMMIT

1. **Operations Execute**
2. **No Errors Detected**
3. **Changes Committed**
4. **Data is Permanent**

ERROR PATH -- ROLLBACK

1. **Operations Execute**
2. **Error Occurs**
3. **Rolled Back**
4. **State Restored**

BEHIND THE SCENES

1. **App Calls Method** @Transactional
2. **Proxy Intercepts**
3. **TX Manager Starts**
4. **Resource Joins TX**
5. **Commit or Rollback**
6. **Returns to Caller**

KEY TAKEAWAY Whether commit or rollback, the lifecycle always ends the same way: resources released and control returned to the caller.

COMMIT OPERATIONS

A commit operation permanently saves all changes made in a transaction when it completes successfully.

1. **Ops Complete**
2. **TX Manager Commits**
3. **Written to DB**
4. **Resources Released**

TransferService.java

```
@Transactional
public TransactionLog transfer(String fromId, String toId,
                               BigDecimal amount) {
  accounts.debit(fromId, amount);
  accounts.credit(toId, amount);
  // Spring commits automatically if no exception is thrown
  return logs.save(new TransactionLog(...));
}
```

- Makes data changes permanent.
- Ensures data consistency.
- Maintains ACID properties.
- Releases database locks.
- Allows other transactions to see the committed data.

KEY TAKEAWAY Commit makes all changes permanent and visible to other transactions -- and releases locks so other work can proceed.

ROLLBACK OPERATIONS

A rollback operation undoes all changes made during a transaction when an error occurs.

1. **Error Occurs**
2. **Manager Rolls Back**
3. **Changes Discarded**
4. **Resources Released**
5. **State Restored**

TransferService.java (Rollback Trigger)

```
@Transactional
public TransactionLog transfer(String fromId, String toId,
                               BigDecimal amount) {
  accounts.debit(fromId, amount);
  accounts.credit(toId, amount);
  if ("ACC-FORCE-FAIL".equals(toId)) {
    throw new IllegalStateException("Forced failure");
  }
  // Any RuntimeException here rolls back debit AND credit.
}
```

- An unhandled exception is thrown during execution.
- Validation fails (e.g. insufficient funds).
- A database error occurs (e.g. deadlock).
- Programmatic rollback via setRollbackOnly().
- A timeout or system-level failure occurs.

KEY TAKEAWAY Rollback ensures partial changes are never saved -- the database returns to a consistent, pre-transaction state.

TRY IT YOURSELF -- EXERCISE 3: ROLLBACK EVIDENCE PLAN

Documentation exercise -- plan exactly what must stay unchanged after a forced ACC-FORCE-FAIL failure.

Goal: document the happy path and the forced-failure path side by side, plus one rule for rejecting unsafe AI-drafted exception handling.

STEP-BY-STEP

Step	What To Do
1 -- Happy path	In notes/rollback-plan.md, describe: MAIN -> LOYALTY updates both balances and writes one TransactionLog row.
2 -- Force fail	Describe: a transfer to ACC-FORCE-FAIL leaves the MAIN balance unchanged and writes no success log row.
3 -- Test idea	Sketch (plan only, no code yet) an automated test that asserts account balances after the forced failure.
4 -- AI caution	Write the rejection rule: reject any AI draft that catches Exception and swallows it inside a @Transactional method.

WHY SWALLOWED EXCEPTIONS BREAK ROLLBACK

Reject This AI-Drafted Pattern

```
@Transactional
public void transfer(String fromId, String toId, BigDecimal amount) {
  try {
    accounts.debit(fromId, amount);
    accounts.credit(toId, amount);
  } catch (Exception e) {
    log.warn("transfer issue", e);  // swallowed -- method returns normally
  }
  // Spring sees no exception propagate out -> COMMITS the partial debit. Atomicity broken.
```

KEY TAKEAWAY TRY IT NOW -- complete Exercise 3 (exercises/exercise-03-rollback-plan.md): happy path and fail path contrasted, no-log-on-fail stated, swallowed-exception rule written.

TRY IT YOURSELF -- EXERCISE 4: TRANSFER PSEUDOCODE (STARTER)

Hands-on exercise -- fill in the TODOs in a starter TransferService sketch; this is a STARTER file, not a finished solution.

Goal: complete a TODO-riddled pseudocode sketch that debits, credits, and rolls back on a forced ACC-FORCE-FAIL failure.

STEP-BY-STEP

Step	What To Do
1 -- Create the file	Create notes/TransferServiceSketch.java in your exercises workspace.
2 -- Fill the TODOs	Replace every blank: @Transactional for one unit of work, IllegalStateException (or RuntimeException) for the forced failure, and amount for both debit/credit calls.
3 -- Self-check	Write one sentence explaining why a caught-and-ignored exception would break atomicity here.
4 -- Reflect	Note that CUS-1001 / CUS-1002 own the seeded accounts -- do not invent a Kafka outbox for this exercise.

FILL THE BLANKS

TransferServiceSketch.java (STARTER -- not a solution)

```
// Sketch only -- not a full Spring project
class TransferService {
    // TODO: Spring annotation for one unit of work
    @_____
    void transfer(String fromId, String toId, long amount) {
        Account from = load(fromId);
        Account to = load(toId);
        if ("ACC-FORCE-FAIL".equals(toId)) {
            // TODO: throw a runtime exception to trigger rollback
            throw new _____("forced failure");
        }
        from.debit(_____);   // TODO: amount
        to.credit(_____);    // TODO: amount
        logSuccess(fromId, toId, amount);
    }
}
```

KEY TAKEAWAY TRY IT NOW -- complete Exercise 4 (exercises/exercise-04-transfer-pseudocode.md) -- a STARTER file, not a solution: every blank filled and the atomicity reflection written.

PROPAGATION TYPES (1 OF 2)

Propagation defines how a transactional method behaves when called from within another transactional method.

It determines whether to join an existing transaction, create a new one, suspend the current one, or throw an exception. REQUIRED is Spring's default.

Type	Behavior	When Called From Another Transaction
REQUIRED (default)	Joins the existing transaction if one exists; otherwise creates a new one.	Runs within the same transaction as the caller.
REQUIRES_NEW	Suspends the existing transaction (if any); creates a new, independent transaction.	Runs in its own transaction, independent of the caller -- caller's transaction is suspended, then resumed.
SUPPORTS	Joins the existing transaction if one exists; otherwise runs without a transaction.	Executes non-transactionally if no transaction exists.

KEY POINTS

- Propagation controls the scope and boundaries of a transaction -- essential when one @Transactional method calls another.
- REQUIRED is Spring's default and by far the most commonly used propagation type.

KEY TAKEAWAY REQUIRED joins or creates a transaction and is the right default for almost every service method.

Transaction Propagation Types

Propagation defines how a transactional method behaves when it is invoked from another transactional method.

Propagation Type	Description	Behavior When Invoked	Execution Flow (Outer → Inner)	Use Case
REQUIRED (Default)	Join existing transaction if one exists; otherwise, create a new transaction.	If a transaction exists, inner method runs in the same transaction. Otherwise, a new transaction is started.	Outer Method (Tx) ----------> Inner Method (Tx) — Same Transaction	Most common use case
REQUIRES_NEW	Always create a new transaction. Suspends any existing transaction.	Existing transaction is suspended and a new independent transaction is started.	Outer Method (Tx) --- Suspended ---> Inner Method (Tx) — New Transaction	Audit logging, notifications, independent tasks
SUPPORTS	Support a current transaction if one exists; otherwise, execute non-transactionally.	If a transaction exists, inner method runs within it. Otherwise, runs without a transaction.	Outer Method (Tx) ----------> Inner Method (◌) — Join if exists, otherwise none	Read-only operations
NOT_SUPPORTED	Execute non-transactionally. Suspends any existing transaction.	Existing transaction is suspended and inner method runs without a transaction.	Outer Method (Tx) --- Suspended ---> Inner Method (◌) — No Transaction	Long-running operations, report generation
MANDATORY	Must run within an existing transaction; throws exception if none.	If no transaction exists, exception is thrown.	Outer Method (Tx) ----------> Inner Method (Tx) — Must exist or fail	Critical operations that require transaction context
NEVER	Must not run within a transaction; throws exception if one exists.	If a transaction exists, exception is thrown.	Outer Method (Tx) ----------> Inner Method (✕) — Must not exist or fail	Operations that must be completely independent
NESTED	Run within a nested transaction if a transaction exists; otherwise, acts like **REQUIRED.**	Creates a savepoint within the existing transaction. Rollback rolls back to savepoint, not the entire transaction.	Outer Method (Tx) ----------> Inner Method (◌) — Savepoint within Tx	Partial rollback within a larger transaction

Key Points

- Propagation controls the relationship between existing and new transactions.
- Choose the right propagation to ensure data integrity and performance.
- Improper use can lead to unexpected commits or rollbacks.
- Understand your use case before selecting propagation type.

Legend

- Tx — Active Transaction
- Tx — New Transaction
- ◌ — No Transaction
- ----> Join / Continue
- ----> New / Independent
- ----- Suspended
- ----> Invalid / Not Allowed
- ◌ — Savepoint (Nested)

Choosing the right propagation type ensures correct transactional behavior, improves system reliability, and maintains data consistency.

PROPAGATION TYPES (2 OF 2)

Four more propagation types -- MANDATORY, NOT_SUPPORTED, NEVER, and NESTED -- plus practical guidance on choosing between them.

Type	Behavior	When Called From Another Transaction
MANDATORY	Must run within an existing transaction; throws an exception if none exists.	Joins the existing transaction; fails with an exception if there is no caller transaction.
NOT_SUPPORTED	Suspends the existing transaction (if any) and runs non-transactionally.	Always executes without a transaction, even if the caller has one.
NEVER	Must not run within a transaction; throws an exception if one exists.	Fails with an exception if there is an existing caller transaction.
NESTED	Runs in a nested transaction within the current one, using savepoints.	Rolls back to the savepoint without affecting the outer transaction.

CHOOSING THE RIGHT PROPAGATION TYPE

- **Use REQUIRES_NEW —** for audit logging, notifications, or independent operations that must persist regardless of the caller's outcome.
- **Use NESTED —** when working with JDBC savepoints for partial rollback within a larger transaction.
- **Use MANDATORY —** to enforce that a method is only ever called from within an already-open transaction.

KEY TAKEAWAY Choose propagation deliberately -- joining, suspending, nesting, and rejecting all trade off differently.

TRY IT YOURSELF -- EXERCISE 5: PROPAGATION WARNINGS

Analysis exercise -- flag common AI and propagation mistakes before Lab 27, including the self-invocation trap.

Goal: list three realistic propagation risks for the transfer flow, then state Lab 27's safe default.

STEP-BY-STEP

Step	What To Do
1 -- List the risks	In notes/propagation-warnings.md, flag three risks: NOT_SUPPORTED used mid-transfer, REQUIRES_NEW used for the log only, and self-invocation bypassing the proxy.
2 -- State the default	Confirm the default propagation REQUIRED on the outer transfer() method is enough for this lab -- no custom propagation needed.
3 -- Note the proxy trap	Write why calling this.transfer(...) from inside the same class may skip the Spring AOP proxy entirely.
4 -- Respect the boundary	State that Lab 27 does not require configuring custom transaction managers -- Spring Boot's defaults suffice.

THE SELF-INVOCATION TRAP

AccountFacade.java (bypasses the proxy)

```
@Service
public class AccountFacade {
  @Transactional
  public void transfer(String fromId, String toId, BigDecimal amount) { /* debit, credit, log */ }

  public void batchTransfer(List<TransferRequest> reqs) {
    for (TransferRequest r : reqs) { this.transfer(r.from(), r.to(), r.amount()); }  // BUG: skips proxy, @Transactional ignored
  }
}
```

KEY TAKEAWAY TRY IT NOW -- complete Exercise 5 (exercises/exercise-05-propagation-warnings.md): three risks listed, REQUIRED default stated, self-invocation warning written.

ISOLATION LEVELS (1 OF 2)

Isolation level defines how visible uncommitted changes are between concurrent transactions.

Higher isolation levels provide stronger consistency but may reduce concurrency and performance.

Isolation Level	Description	Prevents	Performance
READ_UNCOMMITTED	Allows reading uncommitted data from other transactions.	Nothing	Highest (least restrictive)
READ_COMMITTED	Allows reading only committed data; prevents dirty reads.	Dirty reads	High
REPEATABLE_READ	Rows read remain the same for the duration of the transaction.	Dirty + non-repeatable reads	Medium
SERIALIZABLE	Transactions appear to run serially; strictest isolation.	Dirty + non-repeatable + phantom reads	Lowest (most restrictive)
SNAPSHOT (MVCC)	Uses row versioning for a consistent snapshot without locking reads.	All three, optimistically	High (optimistic)

KEY POINTS

- Prevents concurrency problems like dirty reads, non-repeatable reads, and phantom reads.
- Default isolation in most databases (Oracle, SQL Server, PostgreSQL) is READ_COMMITTED.

KEY TAKEAWAY Higher isolation means stronger consistency but lower concurrency -- pick the lowest level you need.

Transaction Isolation Levels

Isolation levels define how visible the changes made by one transaction are to other concurrent transactions.
Higher isolation provides more consistency but can reduce concurrency and performance.

Isolation Level	Description	Dirty Read	Non-Repeatable Read	Phantom Read	Performance	Concurrency	Use Case
READ_UNCOMMITTED	Allows reading uncommitted changes from other transactions.	Yes	Yes	Yes	Highest	Highest	Rarely used. Only in tolerant reporting scenarios.
READ_COMMITTED	Allows reading only committed data. Each read sees the latest committed data.	No	Yes	Yes	High	High	Default in many databases. General business applications.
REPEATABLE_READ	Ensures that data read multiple times cannot change. Prevents non-repeatable reads.	No	No	Yes	Medium	Medium	Financial reporting, consistent reads over multiple queries.
SERIALIZABLE	Provides full isolation. Acts as if transactions are executed serially.	No	No	No	Lowest	Lowest	Critical operations requiring maximum consistency.

Key Points

- Higher isolation = more consistency but lower concurrency.
- Lower isolation = more concurrency but higher risk of anomalies.
- Choose the lowest isolation level that meets your consistency requirements.
- Default levels vary by database (e.g., Oracle: READ_COMMITTED, MySQL: REPEATABLE_READ, SQL Server: READ_COMMITTED).
- You can set isolation per transaction or per connection.

Concurrency Anomalies Prevented by Higher Isolation Levels

Dirty Read (Prevented from READ_COMMITTED)

Transaction T1	Transaction T2
1. Update balance = 1000 → 500 (UNCOMMITTED)	
	2. Read balance = 500 (Dirty Read)
3. ROLLBACK	

READ_COMMITTED and above prevent Dirty Read.

Non-Repeatable Read (Prevented from REPEATABLE_READ)

Transaction T1	Transaction T2
1. Read balance = 1000	
	2. Update balance = 1000 → 800 (COMMIT)
3. Read balance again = 800 (Changed)	

REPEATABLE_READ and above prevent Non-Repeatable Read.

Phantom Read (Prevented by SERIALIZABLE)

Transaction T1	Transaction T2
1. Query: count accounts WHERE balance > 1000 → Result = 5	
	2. Insert new account with balance = 1500 (COMMIT)
3. Query again: count accounts WHERE balance > 1000 → Result = 6 (New row appears)	

SERIALIZABLE prevents Phantom Read.

Legend

- Not Possible
- Possible
- Better (Higher)
- Worse (Lower)
- Higher Concurrency
- Lower Concurrency

Choose the isolation level that balances data consistency needs with application performance.

Best Practice: Use the lowest isolation level that guarantees the consistency your business requires.

ISOLATION LEVELS (2 OF 2)

Concurrency anomalies isolation levels defend against, and the real-world defaults across common databases.

CONCURRENCY ANOMALIES

Dirty Read

Reading data written by another transaction that has not yet been committed. Prevented by READ_COMMITTED and above.

Non-Repeatable Read

Reading the same row twice and getting different results because another transaction modified and committed the data. Prevented by REPEATABLE_READ and above.

Phantom Read

Re-running a query and getting additional rows inserted by another committed transaction. Prevented by SERIALIZABLE and SNAPSHOT.

DEFAULTS ACROSS COMMON DATABASES

Database	Default Isolation	Notes
Oracle Database	READ_COMMITTED	No REPEATABLE_READ option; use SERIALIZABLE if needed.
SQL Server	READ_COMMITTED	Also supports SNAPSHOT isolation via row versioning.
MySQL (InnoDB)	REPEATABLE_READ	A common surprise for developers coming from Oracle/SQL Server.
PostgreSQL	READ_COMMITTED	Also supports SERIALIZABLE with true serializable snapshot isolation.

KEY TAKEAWAY Know your database's default isolation level -- MySQL InnoDB's REPEATABLE_READ default surprises many.

COMMON TRANSACTION PROBLEMS (1 OF 2)

Even with ACID properties, concurrent transactions can cause issues that impact data consistency and system performance.

Problem	Cause	Example	Fix
Dirty Read	Occurs at low isolation levels (e.g. READ_UNCOMMITTED).	T1 transfers $100 but rolls back; T2 already read the uncommitted balance and acted on it.	Use READ_COMMITTED or higher; never expose uncommitted data.
Non-Repeatable Read	Another transaction updates a row and commits between two reads.	T1 reads balance $500; T2 updates it to $600 and commits; T1 reads again and gets $600.	Use REPEATABLE_READ or SERIALIZABLE; lock rows when reading if needed.
Phantom Read	Another transaction inserts or deletes rows matching the query between reads.	T1 queries accounts with balance > $1,000 (5 rows); T2 inserts a new matching row; T1 re-runs the query and gets 6 rows.	Use SERIALIZABLE isolation; use range locks (index locking).

KEY POINTS

- Concurrency issues occur when multiple transactions access the same data at the same time.
- They can lead to inconsistent or incorrect data if not handled properly -- isolation levels and locking help prevent them.

KEY TAKEAWAY Even with ACID guarantees, concurrent transactions can still create real data-visibility problems.

Common Transaction Problems

Poor transaction design or configuration can lead to data inconsistencies, performance issues, and system failures. Understanding common problems helps you prevent and resolve them.

1 Missing or Incorrect @Transactional

Methods that modify data are not annotated, or transactions are misconfigured.

```
public void
  transfer() {
    // update
  }
```

Impact: Partial updates, inconsistent data when errors occur.

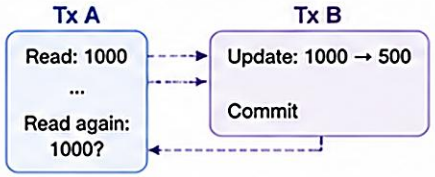
2 Wrong Propagation Type

Improper propagation settings can create unexpected new transactions or commit independently.

Impact: Unexpected commits, rollback does not affect inner transaction.

3 Poor Isolation Level Selection

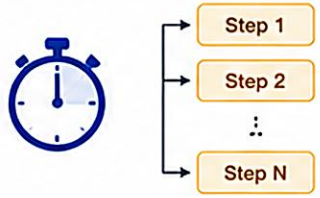
Using low isolation can cause dirty reads, non-repeatable reads, or phantom reads.



Impact: Inconsistent reads and incorrect business decisions.

4 Long-Running Transactions

Transactions that run too long hold locks and consume resources.



Impact: Lock contention, reduced throughput, and timeouts.

5 Swallowing Exceptions

Catching exceptions without rethrowing prevents rollback.

```
try {
  // update
} catch(Exception ex) {
  // ignore
}
```

Impact: Transaction commits even though an error occurred.

6 Self-Invocation Bypass

A method within the same class calls another @Transactional method directly, bypassing the proxy.

Impact: Transactional annotations are ignored, no transaction created.

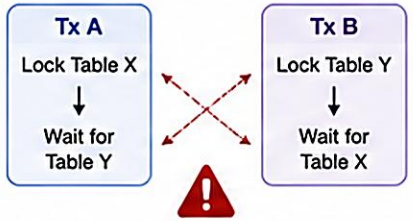
7 N+1 Operations in a Transaction

Executing too many database operations increases duration and lock time.

Impact: Poor performance and higher chance of deadlocks.

8 Deadlocks

Two or more transactions wait for each other's locks indefinitely.



Impact: System hangs and requires manual intervention.

Key Points

- Transactions must be designed with correctness, performance, and concurrency in mind.
- Always handle exceptions properly and let the framework manage rollbacks.
- Choose the right propagation and isolation level for your use case.
- Keep transactions short and focused.
- Test concurrent scenarios to uncover hidden issues.

Best Practices

- Use @Transactional correctly on public methods.
- Choose proper propagation and isolation levels.
- Keep transactions short and failure-safe.
- Handle exceptions and rollback appropriately.
- Monitor and test for concurrency issues.

Well-designed transactions ensure data integrity, system reliability, and optimal performance in enterprise applications.

COMMON TRANSACTION PROBLEMS (2 OF 2)

Lost updates, deadlocks, and long-running transactions -- and the best practices that prevent all five problems.

Problem	Cause	Example	Fix
Lost Update	No locking or version control on the data during update.	T1 and T2 both read balance $500; both update and commit; T2's commit silently overwrites T1's -- final value is T2's, T1's update is lost.	Use locking (SELECT ... FOR UPDATE) or optimistic locking with a version field.
Deadlock	Improper lock order or cyclic dependencies between transactions.	T1 locks Row A and waits for Row B; T2 locks Row B and waits for Row A -- both wait forever.	Ensure consistent lock ordering; keep transactions short; use deadlock timeout and retry.
Long-Running Transaction	Complex queries, network delays, or external calls inside a transaction.	A reporting transaction runs for several minutes and blocks all updates on the affected tables.	Keep transactions short; move long operations outside the transaction; use proper indexing.

BEST PRACTICES TO PREVENT TRANSACTION PROBLEMS

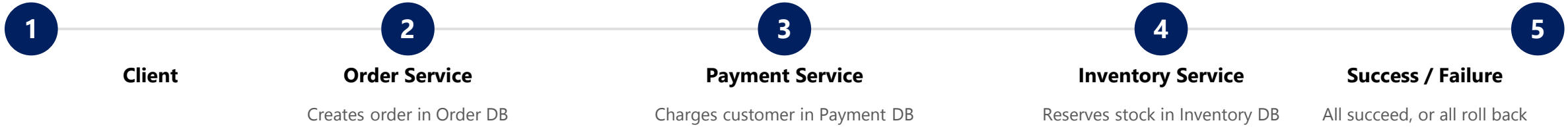
- **Choose the Right Isolation Level**
- **Keep Transactions Short**
- **Use Proper Indexes**
- **Access Data in a Consistent Order**
- **Monitor Long-Running Transactions**
- **Handle Deadlocks and Retries**

KEY TAKEAWAY Lost updates and deadlocks are concurrency risks, not ACID failures -- locking strategy is your defense.

DISTRIBUTED TRANSACTIONS OVERVIEW (1 OF 2)

Distributed transactions span multiple independent resources or services across a network -- a harder, more advanced problem than a single-database transaction.

They ensure that either all participating resources commit or all roll back, maintaining data consistency across the system.



WHY DISTRIBUTED TRANSACTIONS MATTER

- **Data Consistency —** keeps data consistent across multiple independent systems.
- **Business Integrity —** ensures business rules are maintained end-to-end.
- **Complex Workflows —** supports multi-step operations spanning multiple services.
- **Heterogeneous Systems —** works across different databases, platforms, and technologies.

COMMON USE CASES

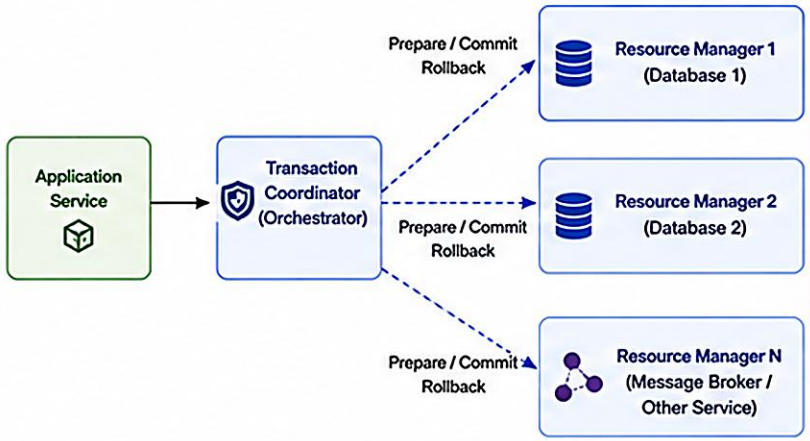
E-Commerce **Banking** **Travel Booking** **Insurance** **Microservices**

KEY TAKEAWAY Distributed transactions guarantee all-or-nothing behavior across services -- but at a much higher cost.

Distributed Transactions Overview

A distributed transaction ensures atomicity and consistency across multiple resources (databases, services, message brokers) that may be on different servers or managed by different systems.

Distributed Transaction Architecture



Two-Phase Commit (2PC) Protocol

Phase 1: Prepare (Voting Phase)

Each resource manager prepares the transaction and responds with YES (can commit) or NO (cannot commit).

Phase 2: Commit (Decision Phase)

If all vote YES → Coordinator sends COMMIT.
If any vote NO → Coordinator sends ROLLBACK.

2PC ensures Atomicity across all participating resources.

Challenges

Performance Overhead
Extra network communication and coordination adds latency.

Coordinator Failure
If the coordinator fails, transactions may be blocked.

Blocking
Resources may remain locked while waiting for decision.

Scalability
2PC does not scale well for a large number of participants.

Heterogeneity
Different systems and capabilities increase complexity.

Common Use Cases

Banking
Fund transfer between multiple accounts in different databases

E-Commerce
Order placement involves DB update, payment gateway, and inventory service

Travel Booking
Flight, hotel, and payment systems across different platforms

Enterprise Integration
Updating multiple systems/databases to keep data consistent

Alternatives to 2PC

Sagas
Break transaction into a sequence of local transactions with compensating actions.

Event-Driven (Eventually Consistent)
Use events to update systems asynchronously. Achieves consistency over time.

TCC (Try-Confirm-Cancel)
Application controls Try, Confirm, Cancel steps explicitly.

Key Points

- Distributed transactions involve multiple independent resource managers.
- Two-Phase Commit (2PC) is the most common protocol.
- Ensures ACID properties across distributed resources.
- Alternatives exist to reduce blocking and improve scalability.

Distributed transactions extend ACID guarantees across multiple resources, ensuring consistency in complex, multi-system environments.

DISTRIBUTED TRANSACTIONS OVERVIEW (2 OF 2)

Two-Phase Commit (2PC), its real challenges, and the alternative Saga pattern.

TWO-PHASE COMMIT (2PC)

- **Phase 1 -- Prepare (Vote):** the coordinator asks all participants to prepare to commit; each responds YES (ready) or NO.
- **Phase 2 -- Commit / Rollback:** if all participants respond YES, the coordinator tells everyone to commit; if any responds NO or times out, everyone rolls back.

CHALLENGES

- **Performance —** extra coordination round-trips increase latency.
- **Single Point of Failure —** coordinator failure can block the transaction.
- **Network Issues —** failures, partitions, or timeouts create uncertainty.
- **Resource Locking —** longer lock duration reduces concurrency.
- **Complexity —** harder to implement, test, and monitor.

Two-Phase Commit (2PC)

Strong consistency using a central coordinator; participants block until the decision arrives.

Saga Pattern

Event-driven approach: a chain of local transactions with compensating actions if a later step fails.

KEY TAKEAWAY Distributed transactions are advanced -- reach for 2PC or Saga once an operation spans independent services.

TRANSACTION BEST PRACTICES (1 OF 2)

Following these best practices builds reliable, consistent, high-performance applications while minimizing failures and contention.

Consistency First

Ensure data consistency and integrity across all operations.

Keep It Short

Reduce locks and improve throughput.

Fail Fast

Validate early to avoid unnecessary rollbacks.

Handle Exceptions

Handle exceptions properly; roll back cleanly.

Design for Scale

Design with scalability and concurrency in mind.

#	Practice	What It Means
1	Define Clear Boundaries	Include only the operations that must succeed or fail together; avoid overly large transactions.
2	Keep Transactions Short	Minimize the time a transaction holds locks and resources; move non-essential processing outside.
3	Choose Appropriate Isolation Level	Use the lowest isolation level that meets your consistency requirements, to improve concurrency.
4	Validate Early	Validate inputs and business rules before starting the transaction to reduce unnecessary rollbacks.
5	Handle Exceptions Properly	Catch and handle exceptions; ensure rollback happens automatically or programmatically when required.

KEY TAKEAWAY Good transaction design ensures data integrity, performance, and system reliability.

TRANSACTION BEST PRACTICES (2 OF 2)

Practices 6-10, and the common pitfalls that violate them in real codebases.

#	Practice	What It Means
6	Use Appropriate Propagation	Choose the right propagation type (REQUIRED, REQUIRES_NEW, etc.) based on business needs.
7	Avoid Long-Held Locks	Avoid user interaction, external calls, or slow operations inside a transaction.
8	Monitor and Tune	Monitor transaction performance, lock waits, and rollbacks; tune isolation levels, indexes, and queries.
9	Test for Concurrency	Test with concurrent users and loads to identify contention, deep rollbacks, and deadlocks early.
10	Log and Audit	Log key transaction events and errors for troubleshooting and audit compliance.

COMMON PITFALLS TO AVOID

- **Long Transactions —** holding locks too long impacts performance and increases deadlock risk.
- **External Calls Inside a Transaction —** API calls, messaging, or file I/O inside a transaction can cause timeouts and partial failures.
- **Wrong Isolation Level —** too high causes blocking; too low causes inconsistent reads.
- **Ignoring Exceptions —** swallowing exceptions prevents proper rollback and leads to inconsistent data.
- **Incorrect Propagation —** using the wrong propagation type can cause unexpected commits or rollbacks.
- **No Monitoring —** not tracking slow transactions, lock waits, or failures leads to unseen issues.

KEY TAKEAWAY Well-designed transactions ensure data integrity and system reliability -- apply these ten practices consistently.

Copilot to generate/validate transaction code

Use GitHub Copilot to quickly generate correct, consistent, and robust transaction code following Spring best practices.

Why use Copilot for transactions?

- Generates boilerplate @Transactional code
- Suggests the right propagation, isolation, and rollback rules
- Helps validate transaction boundaries and exceptions
- Enforces best practices and reduces subtle bugs

Transaction use case

Bank money transfer between two accounts.
All steps must succeed or none should.

1. Withdraw from source account
2. Deposit to destination account
3. Record transfer in ledger

If any step fails, the entire transaction should be rolled back.

Copilot in Action – Generate Transaction Code

1 Write a clear prompt (comment)

```
// Create a service method to transfer money
// from one account to another.
// Use @Transactional with rollback for all
// exceptions.
// Steps: withdraw, deposit, record ledger.
```

2 Copilot suggests the code AI Suggestion

```
@Service
public class AccountService {
   @Autowired
   private AccountRepository accountRepository;
   @Autowired
   private LedgerRepository ledgerRepository;

   @Transactional(rollbackFor = Exception.class)
   public void transfer(Long fromId, Long toId, BigDecimal amount) {
      Account from = accountRepository.findById(fromId)
          .orElseThrow(() -> new NotFoundException("Source account not found"));
      Account to = accountRepository.findById(toId)
          .orElseThrow(() -> new NotFoundException("Destination account not found"));
      // 1. Withdraw
      from.withdraw(amount);
      accountRepository.save(from);
      // 2. Deposit
      to.deposit(amount);
      accountRepository.save(to);
      // 3. Record ledger
      LedgerEntry entry = LedgerEntry.transfer(fromId, toId, amount);
      ledgerRepository.save(entry);
}
```

Validate and Improve with Copilot

Ask Copilot to validate

```
// Review this transaction method.
// Are rollback rules and exception handling correct?
// Suggest improvements if any.
```

Copilot feedback / improvements

```
// Suggestions:
// - Consider using more specific rollback rules
// - Validate amount > 0
// - Handle InsufficientFundsException

@Transactional(rollbackFor = {Exception.class, InsufficientFundsException.class})
public void transfer(Long fromId, Long toId, BigDecimal amount) {
   if (amount.compareTo(BigDecimal.ZERO) <= 0) {
      throw new IllegalArgumentException("Amount must be greater than zero");
   }
   // ... existing logic
}
```

Best practices for transaction code

- Keep transactions short and focused.
- Define transactions in the Service layer.
- Use @Transactional with appropriate rollback rules.
- Avoid calling @Transactional methods from the same class.
- Handle business exceptions and validate inputs.
- Log meaningful information, not sensitive data.

Common Copilot Prompts (Cheat Sheet)

- "Create a @Transactional service method to transfer money between accounts."
- "Add rollbackFor and propagation settings for this method."
- "Validate this transaction code and suggest improvements."
- "Add exception handling for insufficient balance."
- "Create a read-only transaction method to get account balance."

Pro Tip

Be specific in your prompt:
what to do, layers, exceptions, and business rules.

Key @Transactional settings

Attribute	Purpose
propagation	Defines how transactions behave with existing transactions
isolation	Controls visibility of data changes
rollbackFor	Specifies exceptions that trigger rollback
readOnly	Optimizes for read-only operations

Goal: Use Copilot to generate correct, efficient, and reliable transaction code and validate it for robustness and best practices.

</> Less boilerplate. Fewer bugs. Stronger transactions.

TRY IT YOURSELF -- EXERCISE 6: LAB 27 READINESS CHECKLIST

Capstone documentation exercise -- confirm dependencies, workspace path, and evidence folders before starting the graded lab.

Goal: ready the lab27-crm project by confirming what Labs 25-26 already gave you, and what stays deferred to Lab 28.

STEP-BY-STEP

Step	What To Do
1 -- Depends on	In notes/lab27-readiness.md, name what Labs 25 and 26 already established: the layered Service/Repository structure and the dev/H2 configuration profile.
2 -- Path	Write the exact workspace path for the graded project: examples/lab27-crm.
3 -- Evidence folders	Plan a screenshots folder for happy-path and rollback evidence: notes/screenshots/lab-27/.
4 -- Defer	State explicitly what is out of scope: JWT protection of the transfer routes waits for Lab 28.

SCOPE BOUNDARY -- DO NOT BUILD LATER TECHNOLOGY YET

- **Do now —** ACID in CRM language, @Transactional on the service (not the controller), debit+credit+log as one unit of work, ACC-FORCE-FAIL rollback evidence.
- **Do not add yet —** JWT SecurityFilterChain (Lab 28), distributed sagas / Kafka transactions (Week 4), or manual EntityManager commit APIs as the primary style.

KEY TAKEAWAY TRY IT NOW -- complete Exercise 6 (exercises/exercise-06-lab27-readiness.md) right before Lab 27: dependencies named, path written, Lab 28 deferred for JWT.

LAB OVERVIEW -- TRANSACTION MANAGEMENT

Lab 27: Transaction Management with AI Assistance -- Northstar CRM Transfers

BUSINESS SCENARIO

- Agents move funds between CRM sub-accounts (e.g. MAIN -> LOYALTY). A debit without a matching credit is an incident.
- Leadership freezes: all multi-account money movement goes through @Transactional TransferService. Controllers must not own transaction boundaries.
- Demonstrate rollback with a synthetic ACC-FORCE-FAIL destination; document ACID with real before/after balances, not slogans.

LEARNING OBJECTIVES

- Place @Transactional on service methods spanning multiple account updates.
- Explain Spring's proxy-based demarcation and why self-invocation skips it.
- Implement a transfer (debit + credit + log) for CRM accounts.
- Force a mid-operation failure and prove both sides roll back.
- Map Atomicity, Consistency, Isolation, and Durability to observable behavior.

CRM FIXTURES USED THROUGHOUT THE LAB

ID	Owner / Role	Seed Balance	Purpose
ACC-1001-MAIN	CUS-1001 Amina Khan (ACTIVE)	$1,000.00	Happy-path transfer source
ACC-1001-LOYALTY	CUS-1001 Amina Khan	$100.00	Happy-path transfer destination
ACC-1002-MAIN	CUS-1002 Ravi Singh (PROSPECT)	$250.00	Secondary account
ACC-FORCE-FAIL	Synthetic sink	n/a	Triggers the rollback demo

KEY TAKEAWAY Correlation id lab-request-001 anchors every transfer example; ACC-FORCE-FAIL proves rollback, not just the happy path.

LAB TASKS AND DELIVERABLES

Northstar CRM Transfers -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Branch prior CRM to lab27-crm; add spring-boot-starter-data-jpa + H2 persistence.
2	Seed Amina/Ravi accounts (MAIN, LOYALTY) with fixed starting balances.
3	Build AccountRepository, TransactionLogRepository, and the TransactionLog entity.
4	Implement @Transactional TransferService (debit + credit + log, one unit of work).
5	Expose POST /api/transfers; prove the happy path MAIN -> LOYALTY.
6	Demonstrate rollback: transfer to ACC-FORCE-FAIL; prove balances are unchanged.
7	Document ACID with real lab evidence (curls, balances, H2 durability caveat).
8	Automated tests + AI review notes (lab27-001); run mvn test twice.

EXPECTED DELIVERABLES

- @Transactional TransferService + thin controller.
- Seeded accounts + TransactionLog entity.
- Happy-path evidence (MAIN->LOYALTY + correlation).
- ACC-FORCE-FAIL rollback evidence (balances + no log).
- ACID explanation tied to real observations.
- Automated tests proving rollback balances.
- AI review notes or manual equivalent; README runbook.

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/AI Review 10 · Docs 10

KEY TAKEAWAY A decreased balance despite a thrown exception, or @Transactional on the controller, both lose real marks.

MODULE 27 SUMMARY

Transaction management ensures that data remains consistent and reliable even in complex, multi-step operations across system boundaries.

WHAT WE COVERED

Transaction Basics & ACID

A logical unit of work, and the four properties that make it reliable.

@Transactional & Spring Architecture

Declarative management via AOP proxies and PlatformTransactionManager.

Commit, Rollback & Lifecycle

How a transaction begins, executes, and always ends one of two ways.

Propagation & Isolation

Controlling transaction boundaries and concurrent visibility.

Common Problems & Distributed TX

Dirty reads, deadlocks, lost updates -- and the harder multi-service case.

Best Practices

Short, well-bounded transactions with the right isolation and clean rollback.

MODULE FLOW RECAP

1. **ACID Properties**
2. **@Transactional**
3. **Propagation/Isolation**
4. **Best Practices**
5. **Lab: CRM Transfers**

KEY TAKEAWAY Correctly designed transactions protect data consistency and support reliable, enterprise-ready systems.

KNOWLEDGE CHECK (1 OF 3)

Test your understanding of ACID properties, commit, and rollback.

1. Which property ensures that all steps of a transaction are completed successfully or none are?

A. Consistency
B. Atomicity
C. Isolation
D. Durability

2. Which operation makes the changes of a transaction permanent?

A. Rollback
B. Savepoint
C. Commit
D. Flush

3. If a transaction fails, which operation undoes all changes made by that transaction?

A. Commit
B. Revert
C. Rollback
D. Reset

4. Which propagation type joins an existing transaction if one is already in progress?

A. REQUIRED
B. REQUIRES_NEW
C. SUPPORTS
D. NEVER

KEY TAKEAWAY Atomicity is all-or-nothing; COMMIT saves changes permanently; ROLLBACK undoes them entirely.

KNOWLEDGE CHECK (2 OF 3)

Four more questions on isolation levels, propagation, and common concurrency problems.

5. Which isolation level allows dirty reads?

A. READ_COMMITTED
B. REPEATABLE_READ
C. SERIALIZABLE
D. READ_UNCOMMITTED

6. Which propagation type always starts a new transaction, suspending any existing transaction?

A. REQUIRED
B. REQUIRES_NEW
C. MANDATORY
D. SUPPORTS

7. Which isolation level prevents non-repeatable reads?

A. READ_COMMITTED
B. REPEATABLE_READ
C. SERIALIZABLE
D. READ_UNCOMMITTED

8. What is a common problem that can occur with concurrent transactions?

A. Deadlock
B. Cache Miss
C. Memory Leak
D. Null Pointer Exception

KEY TAKEAWAY READ_UNCOMMITTED is least restrictive; REQUIRES_NEW always starts independently; deadlocks are a classic risk.

KNOWLEDGE CHECK (3 OF 3)

Two final questions, plus a bonus challenge question on distributed transactions.

9. Which ACID property ensures that once a transaction is committed, it cannot be changed?

A. Atomicity

B. Consistency

C. Isolation

D. Durability

10. Which of the following is considered a best practice for transaction management?

A. Keep transactions long to avoid frequent commits

B. Swallow exceptions and continue

C. Handle exceptions and roll back when needed

D. Use READ_UNCOMMITTED for better performance always

BONUS CHALLENGE QUESTION (FROM THE CURRICULUM)

"In a distributed system, why is transaction management more complex compared to a single-database system?" -- write your own answer before revealing the model answer below.

- **No single transaction manager —** each service/database has its own local transaction with no shared transaction log.
- **Partial failure across services —** one participant can fail after another has already committed locally.
- **Coordination cost —** patterns like Two-Phase Commit add latency, locking, and a coordinator single point of failure.

KEY TAKEAWAY Durability makes commits permanent; handling exceptions and rolling back correctly is the real best practice.

"Either everything happens, or nothing does
-- that is the promise of a transaction."

MODULE 27 COMPLETE

Transaction Management

You can now design and implement reliable @Transactional boundaries -- with ACID guarantees, sensible propagation and isolation, and clean commit/rollback behavior -- for enterprise Spring applications.